

Para cerrar aquí tienes el archivo `main.dart`.

Este archivo es el **punto de entrada** de la aplicación y cumple una función crítica en ingeniería: orquestar la inicialización de los servicios asíncronos antes de que el motor de renderizado de Flutter intente dibujar el primer frame.

Archivo: `lib/main.dart`

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'services/db_service.dart';
import 'screens/tienda_screen.dart';

void main() async {
  // 1. Garantiza que el motor de Flutter esté vinculado antes de usar servicios
  // nativos
  WidgetsFlutterBinding.ensureInitialized();

  // 2. Inicialización de la persistencia (Isar + Seed de datos)
  // Como es asíncrono, el 'await' impide que la app inicie vacía
  try {
    await DBService.init();
  } catch (e) {
    debugPrint('Error crítico al inicializar la base de datos: $e');
    // En una app real, aquí podrías redirigir a una pantalla de error
  }

  runApp(
    // 3. ProviderScope: El contenedor global de estados de Riverpod
    const ProviderScope(
      child: CarritoApp(),
    ),
  );
}

class CarritoApp extends StatelessWidget {
  const CarritoApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Ingeniería Store Lab',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        colorScheme: ColorScheme.fromSeed(seedColor: Colors.indigo),
        useMaterial3: true,
        // Estilización general de AppBars y Cards para un look profesional
        appBarTheme: const AppBarTheme(
          centerTitle: true,
          backgroundColor: Colors.indigo,
          foregroundColor: Colors.white,
        ),
      ),
    );
  }
}
```

```
    ),  
    ),  
    home: const TiendaScreen(),  
  );  
}  
}
```

Notas importantes:

1. **El lugar del `ProviderScope`:** Es vital recalcar que `ProviderScope` debe envolver a `MaterialApp`. Si intentan usar un provider fuera de este alcance, la app lanzará una excepción.
 2. **Manejo de la asincronía en `main`:** Al convertir el `main` en `async`, Flutter espera a que la base de datos esté lista. Sin esto, los providers intentarían leer de un archivo de Isar que aún no está abierto.
 3. **Configuración Visual (Material 3):** Hemos activado `useMaterial3: true` para que el ejercicio tenga un acabado moderno, acorde a las tendencias actuales de diseño en aplicaciones móviles.
-

Estructura de carpetas del proyecto:

Así debería quedar la estructura de carpetas:

- `lib/`
 - `models/` (`producto.dart`, `carrito_item.dart`)
 - `providers/` (`catalogo_provider.dart`, `carrito_provider.dart`)
 - `services/` (`db_service.dart`)
 - `screens/` (`tienda_screen.dart`)
 - `main.dart`